

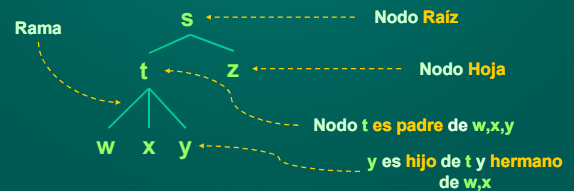
Teoría de la Computabilidad

Módulo 3: Árbol de Derivación Ambigüedad

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Árbol: concepto y terminología

La noción de **árbol** es de uso común en Computación. Un árbol es un **grafo conexo y sin ciclos**. Es representado típicamente “al revés”, con la raíz en el tope y ramas y hojas hacia abajo.

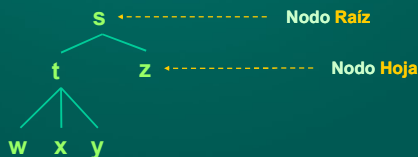


Las hojas (de izq. a der) son la **frontera** del árbol: w,x,y,z

Árbol - definición formal

Un **árbol** es un multidigrafo acíclico tal que:

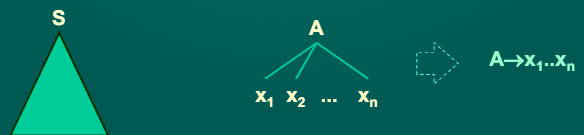
- Hay un único nodo que no tiene arcos incidentes y es llamado **raíz**;
- Si hay mas nodos además de la raíz, éstos tienen exactamente un arco incidente en ellos;
- Cada nodo es alcanzable desde el nodo raíz.



Árbol de derivación

Def.: Un árbol D es un **árbol de derivación** para una gramática $G=(V_n, V_t, S, P)$ si

- La raíz del árbol está **rotulada** como S
- Cada nodo es **rotulado** con un símbolo de $V_n \cup V_t \cup \{\lambda\}$
- Si un nodo interior es **rotulado** A , y sus descendientes directos son **rotulados** $x_1 \dots x_n$, ent. existe en G una producción $A \rightarrow x_1 \dots x_n$

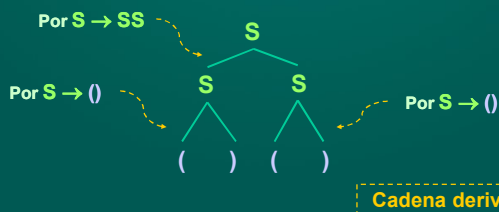


Ejemplo 1

Desarrollar el árbol de derivación para la cadena ‘()
a partir de la gramática:

$G=(V_n, V_t, S, P)$, $V_n=\{S\}$, $V_t=\{(),\}$,

$P=\{S \rightarrow (), S \rightarrow SS, S \rightarrow (S)\}$

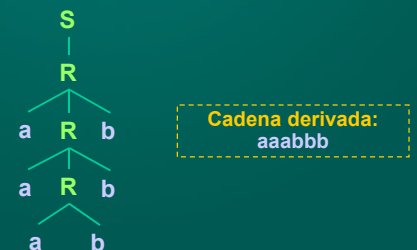


Ejemplo 2

Desarrollar el árbol de derivación para la cadena **aaabbb**, a partir de la gramática:

$G=(V_n, V_t, S, P)$, $V_n=\{S, R\}$, $V_t=\{b, a\}$,

$P=\{S \rightarrow \lambda, S \rightarrow R, R \rightarrow aRb, R \rightarrow ab\}$



Teorema de árboles para GLC

Teo. : Sea $G=(V_n, V_t, S, P)$ una GLC. Entonces para toda cadena $w \in L(G)$, se cumple que

$S \xrightarrow{G^+} w$ sssi existe un árbol de derivación a partir de G con raíz en S cuya frontera es w .

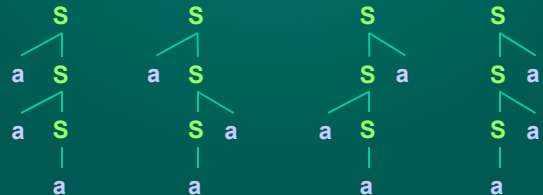
Este teorema nos asegura que **toda cadena de un lenguaje libre de contexto puede derivarse con árboles de derivación**, y que si algo se sigue vía un árbol de derivación, entonces pertenece a algún lenguaje libre de contexto.

¿Qué ocurre si hay más de un árbol de derivación?

Ejemplo: derivando aaa

Sea G la sgte. gramática, con la que queremos derivar $a^3=aaa$

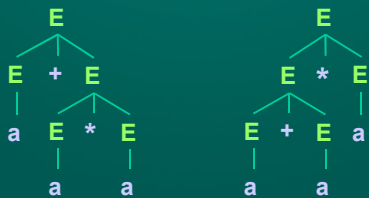
$G=(V_n, V_t, S, P)$, $V_n=\{S\}$, $V_t=\{a\}$,
 $P=\{S \rightarrow aS, S \rightarrow Sa, S \rightarrow a\}$



Ejemplo: derivando a+a*a

Sea G la sgte. gramática, con la que queremos derivar $a+a*a$

$G=(V_n, V_t, S, P)$, $V_n=\{E\}$, $V_t=\{+, *, (,), a\}$,
 $P=\{E \rightarrow E+E, E \rightarrow E^*E, E \rightarrow (E), E \rightarrow a\}$



Gramáticas Ambiguas

En los ejemplos que hemos visto, existen distintos árboles de derivación para una misma cadena w . Este hecho se llama **ambigüedad**.

Def. : Una gramática G libre de contexto se dice **ambigua**, si existe al menos una cadena $w \in L(G)$ tal que existe más de un árbol de derivación con frontera w .

La ambigüedad de una GLC es perjudicial para el procesamiento del lenguaje que genera:

¿cuál es el verdadero resultado de $a+a*a$ en la gramática anterior?

Ejemplo: el 'else' pendiente de Pascal

```

If c1 then
  if c2 then a1
else a2
    
```

Interpretación 1

```

If c1 then
  if c2 then a1
else a2
    
```

Interpretación 2

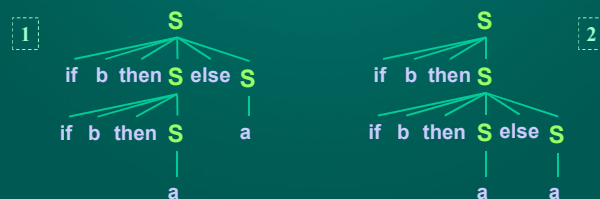
```

If c1 then
  if c2 then a1
else a2
    
```

Una gramática ambigua para el 'else'

Sea G la sgte. gramática

$G=(V_n, V_t, S, P)$, $V_n=\{S\}$, $V_t=\{\text{if, then, else, a, b}\}$,
 $P=\{S \rightarrow \text{if } b \text{ then } S \text{ else } S, S \rightarrow \text{if } b \text{ then } S, S \rightarrow a\}$
 donde $b = \text{exp.booleana}$; $a = \text{sent. de Pascal}$.
 $\Rightarrow \text{if } b \text{ then if } b \text{ then } a \text{ else } a \in L(G)$



Frontera: If b then if b then a else a

Eliminando la ambigüedad de la G anterior

Sea G la sgte. gramática

$G = (V_n, V_t, S_1, P)$, $V_n = \{s_1, s_2\}$, $V_t = \{\text{if, then, else, a, b}\}$,
 $P = \{ s_1 \rightarrow \text{if b then } s_1, \quad s_1 \rightarrow \text{if b then } s_2 \text{ else } s_1$
 $s_1 \rightarrow a, \quad s_2 \rightarrow \text{if b then } s_2 \text{ else } s_2, \quad s_2 \rightarrow a \}$

Para derivar la cadena

if b then (if b then a else a)

se utilizan distintas reglas que para derivar la cadena

if b then (if b then a) **else a**

Cada interpretación de la cadena tiene su propio árbol de derivación en la gramática

13

Ambigüedad en Gramáticas

Nota:

No existe un algoritmo para determinar si una gramática LC es no ambigua.

Sin embargo...

Hay ciertas reglas de producción que son típicamente ambiguas...

14

Lenguajes inherentemente ambiguos

Ejemplo: Una gramática con la producción $A \rightarrow AA$ será ambigua. La subcadena de no terminales AAA tiene dos árboles de derivación, y también los tendrá cualquier cadena w que contenga a AAA



Def.: Si L es un lenguaje libre de contexto, tal que toda gramática que genera L es ambigua, entonces L es **inherentemente ambiguo**

15

Lenguajes inherentemente ambiguos

Ejemplo: Consideremos el lenguaje

$L = \{a^i b^j c^k \mid i=j \text{ o bien } j=k, \text{ para } i, j, k \geq 1\}$.

En este caso, L es inherentemente ambiguo.

¿Por qué razón?

Cualquier GLC que genere L, deberá generar aquellas palabras para las cuales $i=j=k$, y por otra parte deberá considerar $i=j$ solamente y también $j=k$ solamente.

Existirá dos formas de derivar, por ejemplo,

aaabbbccc

16

Lenguajes inherentemente ambiguos

Ejemplo: Consideremos el lenguaje

$L = \{a^i b^j c^k \mid i=j \text{ o bien } j=k, \text{ para } i, j, k \geq 1\}$.

En este caso, L es inherentemente ambiguo.

Intentemos generar L con $G = (\{S, A_1, A_2, B, C\}, \{a, b, c\}, S, P)$,

$P = \{ S \rightarrow A_1 C, \quad A_1 \rightarrow a A_1 b, \quad A_1 \rightarrow ab,$
 $C \rightarrow c C, \quad C \rightarrow c$
 $S \rightarrow A_2 B, \quad A_2 \rightarrow a A_2, \quad A_2 \rightarrow a \quad B \rightarrow b B c$
 $B \rightarrow bc \}$

Tenemos dos formas de producir cadenas con igual cantidad de a y c:

$S \xrightarrow{G} A_1 C \xrightarrow{G^+} a^i b^i c^i$ y también $S \xrightarrow{G} A_2 B \xrightarrow{G^+} a^i b^i c^i$

Utilidad de Formas Normales

• Consideremos los polinomios

$3x^2 + 4x^5 + 2x$ $7x^4 + 4x^5 + 2x$ $2x^5 + 4x$

• Consideremos ahora los mismos polinomios:

$4x^5 + 0x^4 + 0x^3 + 3x^2 + 2x$

$4x^5 + 7x^4 + 0x^3 + 0x^2 + 2x$

$2x^5 + 0x^4 + 0x^3 + 0x^2 + 4x$

Es claro que esta representación es más natural y clara para varios fines (ej: operaciones con polinomios).

Formas Normales de GLC

- Existen algunas simplificaciones a las reglas de producción de una GLC. Esto da lugar a las denominadas “**formas normales**”.
- Tal como su nombre lo sugiere, las mismas proveen una **estructura canónica** en la cual representar una GLC.
- Estas **formas normales** son usadas en distintas demostraciones, y aparecen con frecuencia en libros de texto.
- Razón práctica:** su utilización se traduce en compiladores más eficientes para diversos lenguajes de programación.

Formas Normales de GLC

Destacaremos algunas formas normales (FN):

- Forma Normal de Chomsky
- Forma Normal de Greibach
- Forma Normal de Backus-Naur

Formas Normales de GLC - Chomsky

Def. : Sea G una gramática libre de contexto. Diremos que G respeta la **forma normal (FN) de Chomsky** si toda regla de P es de la forma:

$A \rightarrow BC$ o bien $A \rightarrow a$, donde $A, B, C \in V_n$ y $a \in V_t$

El siguiente teorema relaciona los Lenguajes LC y las GLC en Forma Normal de Chomsky

Teo.: Cualquier Lenguaje libre de contexto puede ser generado por una gramática que respeta la forma normal de Chomsky.

Demostración: sugerida a través del siguiente algoritmo...

Formas Normales de GLC - Chomsky

Algoritmo para transformar GLC G a FN Chomsky

Entrada: $G=(V_n, V_t, S, P)$,

Salida: $G'=(V_n', V_t, S, P')$ en FN Chomsky

Mientras existen producciones cuyo lado derecho no respeta la FN Chomsky hacer

- Reemplazar cada producción $A \rightarrow B$ tales que $B \rightarrow w_1|w_2|\dots|w_n$ por la producción $A \rightarrow w_1|w_2|\dots|w_n$.
- Para cada producción, si su lado derecho tiene **dos o más símbolos** y contiene algún terminal a , reemplace las apariciones de a por un nuevo no terminal N . Agregue la regla $N \rightarrow a$. Es decir, para un no terminal N tal que $N \notin V_n$, $V_n' = V_n \cup \{N\}$, y $P' = P \cup \{N \rightarrow a\}$
- Para cada producción $B \rightarrow C_1C_2\dots C_n$, donde $n > 2$, reemplazar la producción por $B \rightarrow C_1D$, y $D \rightarrow C_2\dots C_n$, que introducen un nuevo no terminal D .

Ejemplos de los pasos anteriores

- Reemplazar cada producción $A \rightarrow B$ tales que $B \rightarrow w_1|w_2|\dots|w_n$ por la producción $A \rightarrow w_1|w_2|\dots|w_n$.

Ejemplo: $A \rightarrow B$, $B \rightarrow a | b | c \implies A \rightarrow a | b | c$

- Para cada producción, si su lado derecho tiene **dos o más símbolos** y contiene algún terminal a , reemplace las apariciones de a por un nuevo no terminal N . Agregue la regla $N \rightarrow a$. Es decir, para un no terminal N tal que $N \notin V_n$, $V_n' = V_n \cup \{N\}$, y $P' = P \cup \{N \rightarrow a\}$

Ejemplo: $A \rightarrow pBq \implies A \rightarrow PBQ$, $P \rightarrow p$, $Q \rightarrow q$,

- Para cada producción $B \rightarrow C_1C_2\dots C_n$, donde $n > 2$, reemplazar la producción por $B \rightarrow C_1D$, y $D \rightarrow C_2\dots C_n$, que introducen un nuevo no terminal D .

Ejemplo: $A \rightarrow BCDE \implies A \rightarrow BC_1$, $C_1 \rightarrow CDE$

Ejemplo: aplicando el algoritmo FNChomsky

$G=(V_n, V_t, S, P)$, $V_n=\{S, T\}$, $V_t=\{a, b\}$,
 $P=\{S \rightarrow aSa, T \rightarrow b\}$

Reescribimos esto para simplificar la lectura:

$P=\{S \rightarrow aSa, S \rightarrow T, T \rightarrow b\}$

Aplicando el algoritmo anterior, resulta

$P_1=\{S \rightarrow aSa, S \rightarrow b\}$ (aplico Paso 1)

$P_2=\{S \rightarrow ASA, S \rightarrow b, A \rightarrow a\}$ (aplico Paso 2)

$P_3=\{S \rightarrow AB, S \rightarrow b, B \rightarrow SA, A \rightarrow a\}$ (Paso 3)

Ejemplo: aplicando el algoritmo FNChomsky

Obtenemos:

$G'=(V_n', V_t, S, P')$, $V_n'=\{S, A, B\}$, $V_t=\{a, b\}$,
 $P'=\{S \rightarrow AB, S \rightarrow b, B \rightarrow SA, A \rightarrow a\}$ o equiv.
 $P'=\{S \rightarrow AB \mid b, B \rightarrow SA, A \rightarrow a\}$

Ventajas de FNChomsky:

- Todo árbol de derivación es de tipo binario
- Toda cadena de longitud n es derivable en $2n-1$ pasos.

Estos elementos son de importancia teórica y práctica (ej: programación de compiladores).

Forma Normal de Greibach

Def. : Sea $G = (V_n, V_t, S, P)$ una gramática libre de contexto. Diremos que G respeta la **forma normal de Greibach** (FN Greibach) si toda regla de P es de la forma:

$A \rightarrow a Z$, donde $A \in V_n$, $a \in V_t$, y $Z \in V_n^*$

El siguiente teorema relaciona los Lenguajes LC y las GLC en Forma Normal de Greibach

Teo.: Cualquier Lenguaje libre de contexto puede ser generado por una gramática que respeta la forma normal de Greibach.

Demostración: queda como ejercicio.

Opción: suponer que la GLC está en FNChomsky.

Sheila Greibach



Nacida en New York (1939), obtuvo su doctorado en 1963 en la Univ. de Harvard.

Actualmente trabaja en el Dep. de Cs. de la Computación de la UCLA (EEUU). Email:

greibach@cs.ucla.edu

Algoritmo [esquema] para GLC $G \rightarrow$ FN Greibach

Entrada: $G=(V_n, V_t, S, P)$,

Salida: $G'=(V_n', V_t, S, P')$ en FN Greibach

- 1) Eliminar la recursión a izquierda (ej: $A \rightarrow Aa$) sustituyendo cada sublenguaje generado recursivamente por un conjunto de reglas equivalente, pero sin recursión.
- 2) Agregar nuevas reglas de reescritura y no terminales para llevar las reglas existentes a la forma de Greibach:
 - a) Reemplazar $A \in V_n$ en cada $\alpha \rightarrow \beta$, de a una por vez, para introducir terminales a izquierda.
 - b) Reemplazar cada $a \in V_t$ que no está en la parte izquierda de β por nuevos $A' \in V_n$ (esto deja solo no terminales a derecha).

Ejemplo: aplicando FNGreibach (1)

$G=(V_n, V_t, S, P)$, $V_n=\{S, A, B\}$, $V_t=\{a, b, c\}$,
 $P=\{S \rightarrow AB \mid B, A \rightarrow Aa \mid b, B \rightarrow Ab \mid c\}$

Reescribamos para simplificar lectura:

$P=\{S \rightarrow AB, S \rightarrow B, A \rightarrow Aa, A \rightarrow b, B \rightarrow Ab, B \rightarrow c\}$

Esta gramática tiene la regla con recursión a izq.:

$A \rightarrow Aa$

Esta regla junto con " $A \rightarrow b$ " genera secuencias de la forma ba^*

Puede transformarse en una secuencia equivalente reemplazando las reglas anteriores por

$\{A \rightarrow b, A \rightarrow bD, D \rightarrow aD, D \rightarrow a\}$

Se tiene así:

Ejemplo: aplicando FNGreibach (2)

$P'=\{S \rightarrow AB, S \rightarrow B, A \rightarrow b, A \rightarrow bD, D \rightarrow aD, D \rightarrow a, B \rightarrow Ab, B \rightarrow c\}$

Ahora es necesario realizar reemplazos para dejar los terminales del lado izquierdo en cada parte derecha de una regla.

Podemos comenzar reemplazando el no terminal B en la parte derecha de una regla. Podemos comenzar reemplazando B en la parte derecha de las reglas de P' . En este caso solo es estrictamente necesario en $S \rightarrow B$, obteniendo:

$P''=\{S \rightarrow AB, S \rightarrow Ab, S \rightarrow c, A \rightarrow b, A \rightarrow bD, D \rightarrow aD, D \rightarrow a, B \rightarrow Ab, B \rightarrow c\}$

Ej.: aplicando FNGreibach (3)

$P'' = \{S \rightarrow AB, S \rightarrow Ab, S \rightarrow c, A \rightarrow b, A \rightarrow bD, D \rightarrow aD, D \rightarrow a, B \rightarrow Ab, B \rightarrow c\}$

Análogamente, podemos reemplazar las apariciones del no terminal A. Obtenemos:

$P''' = \{S \rightarrow bB, S \rightarrow bDB, S \rightarrow bb, S \rightarrow bDb, S \rightarrow c, D \rightarrow aD, D \rightarrow a, B \rightarrow bb, B \rightarrow bDb, B \rightarrow c\}$

Ej.: aplicando FNGreibach (4)

$P''' = \{S \rightarrow bB, S \rightarrow bDB, S \rightarrow bb, S \rightarrow bDb, S \rightarrow c, D \rightarrow aD, D \rightarrow a, B \rightarrow bb, B \rightarrow bDb, B \rightarrow c\}$

Pero no toda regla es de la forma $A \rightarrow aZ, Z \in V_n^*$

Entonces finalmente introducimos una nueva regla $E \rightarrow b$ para respetar la FNGreibach:

$P'''' = \{S \rightarrow bB, S \rightarrow bDB, S \rightarrow bE, S \rightarrow bDE, S \rightarrow c, D \rightarrow aD, D \rightarrow a, B \rightarrow bE, B \rightarrow bDE, B \rightarrow c, E \rightarrow b\}$

$P'''' = \{S \rightarrow bB|bDB|bE|bDE|c, D \rightarrow aD|a, B \rightarrow bE|bDE|c, E \rightarrow b\}$

Ventaja de FNGreibach: se conoce que cada cadena de longitud n es derivable en n pasos.

Forma Normal de Backus-Naur (BNF)

La Forma Normal de Backus-Naur (*Backus-Naur Normal Form = B.N.F*) recurre a metasímbolos y no terminales (llamados metavariabes), que no pertenecen al lenguaje.

La descripción del lenguaje se hace con reglas:

Metavariabes $\leftrightarrow$ no terminales de G

Metasímbolo ::= $\leftrightarrow$ símbolo $\rightarrow$

Metasímbolo | $\leftrightarrow$ opciones alternativas en G

Metavariabes se distinguen con símbolos $\langle$ y $\rangle$.

Forma Normal de Backus-Naur (BNF) - Ejemplo

Definición de FOR de Pascal con BNF

$\langle \text{sent.FOR} \rangle ::=$

$\langle \text{cláusulaFOR} \rangle \langle \text{sentencia} \rangle$

$\langle \text{clausulaFOR} \rangle ::=$

FOR $\langle \text{ident} \rangle := \langle \text{listaFOR} \rangle$ do

$\langle \text{listaFOR} \rangle ::=$

$\langle \text{exp_arit} \rangle$ TO $\langle \text{exp_arit} \rangle$ |

$\langle \text{exp_arit} \rangle$ DOWNT0 $\langle \text{exp_arit} \rangle$

Forma Normal de Backus-Naur (BNF) - Ejemplo

Definición de IF-THEN-ELSE con BNF

$\langle \text{sentIF} \rangle ::= \text{IF} \langle \text{expr} \rangle \text{ THEN} \langle \text{sent} \rangle$ |

$\text{IF} \langle \text{expr} \rangle \text{ THEN} \langle \text{sent} \rangle \text{ ELSE} \langle \text{sent} \rangle$

$\langle \text{expr} \rangle ::= \langle \text{expr_simple} \rangle$ |

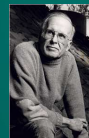
$\langle \text{expr_simple} \rangle \langle \text{op_rel} \rangle \langle \text{expr_simple} \rangle$

$\langle \text{op_rel} \rangle ::= < | > | = | <= | >= | <> | \text{in}$

$\langle \text{expr_simple} \rangle ::= \dots$

John Backus – Peter Naur

John
Backus



Peter
Naur



En 1959 desarrollaron la notación BNF

John Backus: nació el 3/12/1923, en Philadelphia (USA). Falleció el 17/03/2007. Inventó el lenguaje Fortran y el lenguaje funcional FP. Premio Turing 1977.

Peter Naur: nació el 25/10/1928 en Dinamarca. Falleció el 3 de enero de 2016. Fue uno de los diseñadores del lenguaje ALGOL, precursor de la programación estructurada en alto nivel. Premio Turing 2005.

BNF y su relación con GLC

BNF fue desarrollada en paralelo con las GLC de Chomsky.

La notación BNF es equivalente a usar Diagramas de Conway (tradicionales en Pascal).

Ejercicio: demuestre esta última equivalencia.

37

Resumen

- Arbol de Derivación y su relación con GLC
- Gramáticas ambiguas. Lenguajes ambiguos
- Formas Normales de Chomsky, Greibach, Backus-Naur.

38